

wpipe at the center of your pipelines

Fire & Forget: tasks that don't block your pipeline

The context

Not every task in a flow needs to be awaited. Telemetry, notifications, an email send, or an internal sync can run without blocking the main result. In most orchestrators, "launching something non-blocking" is a battle: queues, workers, brokers, supervisors. With wpipe it's a single class: `Background``.

Why engineers shift to wpipe

- 📦 Send metrics or logs to an external service.
- 🔔 Notifications that must not delay business.
- 🧹 Cleanup and post-execution auxiliary tasks.
- Emails / webhooks where latency doesn't matter.

Code-first

Pipelines are Python: versionable, testable, reviewable in Git.

Checkpoints

State persisted to SQLite WAL at every step; resume, don't restart.

Independently lean

<50MB RAM. No broker, no daemon, no metadata server.

What it looks like

```
from wpipe import Pipeline, step

@step(name="my_step", retry_count=3, retry_delay=2, checkpoint=True)
def my_step(data):
    return {"result": process(data["input"])}

pipe = Pipeline(pipeline_name="my_pipeline", tracking_db="pipeline.db")
pipe.set_steps([my_step])
pipe.run({"input": [...]})

# Python-native orchestration: checkpoints, retries, self-documenting.
```

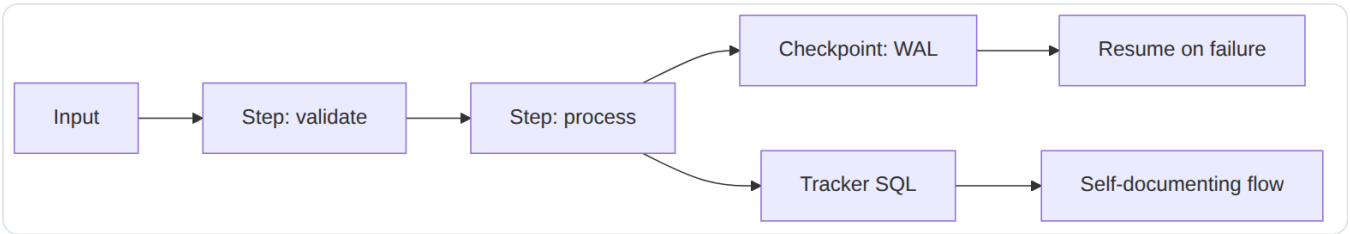
At a glance

Aspect	Legacy / wpipe	wpipe
Definition	Config/DSL or canvas	Python code
State	Scattered / none	SQLite WAL checkpoints

Aspect	Legacy / wpipe	wpipe
Observability	Partial	Tracker SQL + dashboard
Footprint	Heavy	<50MB RAM

Under the hood

The flow behind the post, as wpipe renders it from code.



+117k downloads

A community of efficiency-first developers.

<50MB RAM

Fits containers, VMs, and Raspberry Pi-class hardware.

SQLite WAL

Industrial durability without a database to administer.

Next step

wpipe is a Python library — try it in your next pipeline:

```
pip install wpipe
```

Steps stay functions you already understand. Docs, diagrams, and dashboards follow. Explore **wpipe** workflows at wpipe.readthedocs.io.